



School: Campus:
Academic Year: Subject Name: Subject Code:
Semester: Program: Branch: Specialization:
Date:

Applied and Action Learning

(Learning by Doing and Discovery)

Name of the Experiment : Decentralized Identity – DID and Credential Demo

* Coding Phase: Pseudo Code / Flow Chart / Algorithm

ALGORITHM:

1. Start
2. Open Remix IDE and create a Solidity file named `DIDIdentity.sol`.
3. Import OpenZeppelin's Ownable.sol for ownership and access control.
4. Write a smart contract that:
 - Registers a new Decentralized Identifier (DID) for a user.
 - Allows the contract owner to issue Verifiable Credentials (VCs) linked to DIDs.
 - Allows users to fetch their registered credentials from the blockchain.
5. Compile the smart contract using the Solidity compiler (v0.8.x) in Remix. And Connect MetaMask Wallet and select Injected Provider – MetaMask. After that Deploy the contract on the Sepolia Testnet. Then Copy the contract address and ABI generated after deployment.
6. Create a React frontend using create-react-app.
7. Use Ethers.js to connect the frontend with the deployed DID contract.
8. Design a simple interface where:
 - Users can register their DID.
 - The contract owner can issue credentials.
9. Test the working of DID registration and credential retrieval through the frontend.
10. End

* Software Used:

1. Remix IDE – for writing and compiling Solidity smart contracts.
2. MetaMask Wallet – for blockchain transactions and Sepolia deployment.
3. Sepolia Testnet – for testing the DID smart contract.
4. React.js – for creating the user-friendly decentralized identity frontend.
5. Ethers.js – for interaction between frontend and smart contract.
6. Brave/Chrome Browser – for running Remix IDE and DApp interface.

* Implementation Phase: Final Output (no error)

1. Open the Remix IDE and paste the DIDIdentity.sol smart contract.

Smart Contract Code:-

```
// SPDX-License-Identifier: MIT
```

```
pragma solidity ^0.8.0;
```

```
// ☒ Importing OpenZeppelin Ownable contract for admin control
```

```
import "https://github.com/OpenZeppelin/openzeppelin-  
contracts/blob/master/contracts/access/Ownable.sol";
```

```
/*
```

```
Title: Decentralized Identity and Credential Demo
```

```
Author: (Your Name)
```

```
Description:
```

```
A simple Solidity contract to register Decentralized Identifiers (DIDs)  
and issue Verifiable Credentials (VCs) to registered users.
```

```
*/
```

```
contract DIDIdentity is Ownable {
```

```
// Structure for user DID and Credential
```

```
struct Identity {
```

```
    string did;          // DID string (e.g., did:ethr:0x1234...)
```

```
    string credential;    // Credential issued to this DID
```

```
    bool registered;      // Whether user is registered or not
```

```
}
```

```
// Mapping user address to identity details
```

```
mapping(address => Identity) private identities;
```

```
// Events for logging actions
```

```
event DIDRegistered(address indexed user, string did);
```

```
event CredentialIssued(address indexed user, string credential);
```

```
// ☒ Register a new Decentralized Identifier (DID)
```

```
function registerDID(string memory _did) external {
```

```
    require(!identities[msg.sender].registered, "DID already registered");
```

```
    identities[msg.sender].did = _did;
```

```
    identities[msg.sender].registered = true;
```

```
    emit DIDRegistered(msg.sender, _did);
```

```
}
```

```
// ☒ Issue a Verifiable Credential (only Owner)
```

```
function issueCredential(address _user, string memory _credential) external onlyOwner {
    require(identities[_user].registered, "User DID not registered");
    identities[_user].credential = _credential;
    emit CredentialIssued(_user, _credential);
}
```

// ☒ Retrieve a user's DID and Credential

```
function getIdentity(address _user)
    external
    view
    returns (string memory did, string memory credential)
{
    Identity memory id = identities[_user];
    require(id.registered, "No DID found for this user");
    return (id.did, id.credential);
}
}
```

2. Select the Solidity Compiler version 0.8.x and compile the contract.
3. If any import errors appear for OpenZeppelin, correct them using the GitHub import path:
import "https://github.com/OpenZeppelin/openzeppelin-
contracts/blob/master/contracts/access/Ownable.sol";
4. Ensure that the contract compiles successfully with no syntax or logic errors.
5. Once compiled, Remix generates the ABI and Bytecode automatically.
6. Connect MetaMask and switch to the Sepolia Test Network.
7. Deploy the smart contract using Injected Provider - MetaMask and confirm the transaction.

1. After successful deployment, copy the Deployed Contract Address.
2. Interact with your deployed contract:
 - Call the registerDID(address, string) function to register user DIDs.
 - Call the issueCredential(address, string) function to issue credentials to a registered DID.
 - Retrieve credentials using the getCredential(address) function.
3. The frontend (React + Ethers.js) connects to this smart contract using the ABI and contract address.
4. Users can:
 - View their DID and issued credential details on the interface.
 - Verify that data is stored on-chain.
5. The final output shows the successfully registered DIDs and their credentials without any runtime errors.

Observation:

1. Decentralized Identity (DID) ensures users own and control their identity data without central authority.
2. Smart contracts enable trustless credential issuance on blockchain networks.
3. Ethers.js effectively connects the React frontend with the deployed DID contract.
4. MetaMask provides a secure way for users to authenticate and sign transactions.
5. The demonstration successfully shows how DIDs and credentials can be verified on-chain, ensuring transparency and authenticity.
6. This experiment highlights the future of digital identity management using blockchain-based decentralized identifiers.

ASSESSMENT

Rubrics	Full Mark	Marks Obtained	Remarks
Concept	10		
Planning and Execution/ Practical Simulation/ Programming	10		
Result and Interpretation	10		
Record of Applied and Action Learning	10		
Viva	10		
Total	50		

Signature of the Student:

Name :

Regn. No. :

Signature of the Faculty:

Page No.....

**As applicable according to the experiment.
Two sheets per experiment (10-20) to be used.*